

1. Introduction to Decision Making

Decision-making is one of the most important concepts in programming.

Programs often need to:

- Check conditions
- Make decisions
- Perform different actions

Conditional statements help programs become intelligent and interactive.

Example

```
age = 20
```

```
if age >= 18:  
    print("Eligible to Vote")
```

2. What are Nested Conditions?

A condition inside another condition is called a **nested condition**.

In Python, nested conditions are created by placing one if statement inside another if statement.

Basic Syntax

```
if condition1:  
  
    if condition2:  
        statements
```

Explanation

- First condition is checked
 - If first condition is True
 - Then second condition is checked
 - If second condition is also True
 - Statements execute
-

3. Why Nested Conditions are Important

Nested conditions are useful when:

- Multiple conditions must be checked
 - One condition depends on another
 - Complex decisions are required
-

Real-Life Example

ATM Machine:

1. Check ATM card
2. Check PIN
3. Check account balance
4. Allow withdrawal

Each step depends on previous conditions.

4. Working of Nested Conditions

Example

age = 22

citizen = True

if age >= 18:

 if citizen:

 print("Eligible to Vote")

Output

Eligible to Vote

Explanation

Step 1:

Check age >= 18

- True → Go inside first if block

Step 2:

Check citizen

- True → Print message
-

5. Flow of Execution

Start



Check Condition 1



True?



Check Condition 2



True?



Execute Statements



Stop

6. Difference Between Simple and Nested Conditions

Simple Condition	Nested Condition
------------------	------------------

Single condition	Multiple conditions
------------------	---------------------

Easy logic	Complex logic
------------	---------------

One decision level	Multiple decision levels
--------------------	--------------------------

Simple programs	Real-world systems
-----------------	--------------------

7. Real-Life Examples of Nested Conditions

Nested conditions are used in:

- ATM systems
- Login systems
- Banking software
- E-commerce websites
- Student result systems

- AI systems
 - Security verification systems
-

8. Problem Solving in Programming

Problem solving means:

- Understanding problems
- Breaking them into smaller steps
- Writing logical solutions

Programming is mainly about solving problems efficiently.

Why Problem Solving is Important

Good problem-solving skills help programmers:

- Write efficient programs
 - Build real-world applications
 - Improve logical thinking
 - Reduce errors
 - Optimize solutions
-

9. Steps for Problem Solving

Step 1: Understand the Problem

Read carefully:

- What is required?
 - What are inputs?
 - What should be output?
-

Example

Problem:

Check whether a student passed or failed.

Input:

Marks

Output:
Pass or Fail

Step 2: Break into Small Steps

1. Take marks input
 2. Check marks
 3. Display result
-

Step 3: Create Algorithm

Example:

1. Start
 2. Input marks
 3. If marks ≥ 35
 - Print Pass
 4. Else
 - Print Fail
 5. Stop
-

Step 4: Convert into Code

```
marks = int(input("Enter marks: "))
```

```
if marks >= 35:
```

```
    print("Pass")
```

```
else:
```

```
    print("Fail")
```

10. Algorithmic Thinking

Algorithmic thinking means solving problems step-by-step logically.

It helps programmers:

- Design efficient solutions
- Improve coding skills

- Handle complex systems
-

Example of Algorithmic Thinking

Problem:

Check largest number.

Steps:

1. Input two numbers
 2. Compare both
 3. Display greater number
-

Program

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))
```

```
if a > b:  
    print("A is Greater")  
else:  
    print("B is Greater")
```

11. Dry Run Technique

A dry run means manually checking program execution step-by-step.

It helps understand:

- Variable changes
 - Loop execution
 - Condition flow
-

Example

```
num = 10
```

```
if num > 5:
```

```
    if num < 20:  
        print("Valid Number")
```

Dry Run Table

Step	Condition	Result
1	num > 5	True
2	num < 20	True
3	Print Statement Executed	

12. Programs Using Nested Conditions

Program 1: Voting Eligibility

```
age = int(input("Enter age: "))
citizen = input("Are you citizen? (yes/no): ")
```

```
if age >= 18:
```

```
    if citizen == "yes":
        print("Eligible to Vote")
```

```
    else:
        print("Citizen Required")
```

```
else:
    print("Under Age")
```

Program 2: Login System

```
username = input("Enter username: ")
password = input("Enter password: ")
```

```
if username == "admin":
```

```
    if password == "1234":
        print("Login Successful")
```

```
    else:
        print("Wrong Password")
```

```
else:
    print("Invalid Username")
```

Program 3: Student Grade System

```
marks = int(input("Enter marks: "))
```

```
if marks >= 35:
```

```
    if marks >= 75:
```

```
        print("Distinction")
```

```
    else:
```

```
        print("Pass")
```

```
else:
```

```
    print("Fail")
```

Program 4: ATM Withdrawal System

```
balance = 5000
```

```
amount = int(input("Enter withdrawal amount: "))
```

```
if amount <= balance:
```

```
    if amount > 0:
```

```
        print("Transaction Successful")
```

```
    else:
```

```
        print("Invalid Amount")
```

```
else:
```

```
    print("Insufficient Balance")
```

13. Mini Real-World Problems

Problem 1: Age Verification

```
age = int(input("Enter age: "))
```

```
if age >= 18:
```

```
    if age <= 60:
```

```
        print("Working Age Group")
```



```
else:  
    print("Senior Citizen")
```

```
else:  
    print("Minor")
```

Problem 2: Exam Qualification

```
attendance = int(input("Enter attendance percentage: "))  
fees_paid = input("Fees Paid? (yes/no): ")
```

```
if attendance >= 75:
```

```
    if fees_paid == "yes":  
        print("Eligible for Exam")
```

```
    else:  
        print("Pay Fees First")
```

```
else:  
    print("Low Attendance")
```

14. Common Errors in Nested Conditions

1. Wrong Indentation

Python uses indentation to define blocks.

Wrong:

```
if age >= 18:  
if citizen:  
    print("Eligible")
```

Correct

```
if age >= 18:  
  
    if citizen:  
        print("Eligible")
```

2. Missing Colon

Wrong:

if age >= 18

Correct:

if age >= 18:

3. Incorrect Condition Logic

Wrong conditions may produce incorrect output.

15. Advantages of Nested Conditions

- Helps solve complex problems
 - Enables multi-level decisions
 - Improves program intelligence
 - Useful in real-world applications
 - Makes systems more secure
-

16. Real-Life Applications

Nested conditions are used in:

- Banking software
- AI systems
- Login authentication
- Payment gateways
- Hospital management systems
- School result systems
- Security systems